

1. Harris Corner Detection Analysis

Question

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

Python Code

```
import cv2

import numpy as np

import tkinter as tk

from tkinter import filedialog

# Select image

root = tk.Tk()

root.withdraw()

image_path = filedialog.askopenfilename(

    title="Select Image for Harris Corner Detection",

    filetypes=[

        ("Image Files", "*.jpg *.jpeg *.png *.bmp")

    ]

)

if image_path == "":

    print("No image selected.")

    exit()

print("Selected image:", image_path)
```

```
# Read image
img = cv2.imread(image_path)

if img is None:
    print("ERROR: Image could not be loaded.")
    exit()

print("Image loaded successfully!")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Convert to float32
gray_float = np.float32(gray)

# Harris Corner Detection
corners = cv2.cornerHarris(
    gray_float,
    blockSize=2,
    ksize=3,
    k=0.04
)

# Dilate for better visibility
corners = cv2.dilate(corners, None)

# Copy image
result = img.copy()
```

```
# Mark detected corners
threshold = 0.01 * corners.max()

result[corners > threshold] = [0, 0, 255]

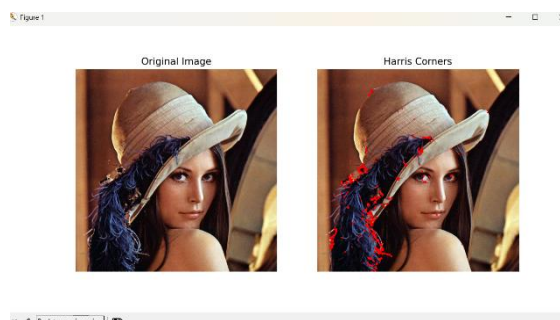
# Count corners
corner_count = np.sum(corners > threshold)

print("Number of detected corners:", corner_count)

# Display
cv2.imshow("Original Image", img)
cv2.imshow("Harris Corner Detection", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result

The Harris algorithm detects important corner points in the image and marks them in **red**.

2. Effect of Noise on Corner Detection

Question

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

Python Code

```
import cv2

import numpy as np

import tkinter as tk

from tkinter import filedialog

# -----

# Select Image

# -----

root = tk.Tk()

root.withdraw()

image_path = filedialog.askopenfilename(

    title="Select Image for Noise Experiment",

    filetypes=[

        ("Image Files", "*.jpg *.jpeg *.png *.bmp")

    ]

)

if image_path == "":

    print("No image selected.")

    exit()
```

```
print("Selected image:", image_path)

# -----
# Read Image
# -----

img = cv2.imread(image_path)

if img is None:
    print("ERROR: Image could not be loaded.")
    exit()

print("Image loaded successfully!")

# -----
# Convert to Grayscale
# -----

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# -----
# Add Gaussian Noise
# -----

noise = np.random.normal(
    0,
    25,
    gray.shape
)

noisy = gray.astype(np.float32) + noise
```

```
noisy = np.clip(
    noisy,
    0,
    255
).astype(np.uint8)

# -----
# Harris Function
# -----

def detect_harris(image):

    image_float = np.float32(image)

    corners = cv2.cornerHarris(
        image_float,
        blockSize=2,
        ksize=3,
        k=0.04
    )

    corners = cv2.dilate(corners, None)

    result = cv2.cvtColor(
        image,
        cv2.COLOR_GRAY2BGR
    )

    threshold = 0.01 * corners.max()
```

```

result[corners > threshold] = [
    0, 0, 255
]

count = np.sum(corners > threshold)

return result, count

# -----
# Original Image Harris
# -----
original_result, original_count = detect_harris(gray)

# -----
# Noisy Image Harris
# -----
noisy_result, noisy_count = detect_harris(noisy)

# -----
# Noise Reduction
# -----
filtered = cv2.GaussianBlur(
    noisy,
    (5, 5),
    1
)

# -----

```

```
# Harris After Filtering
# -----
filtered_result, filtered_count = detect_harris(
    filtered
)
```

```
# -----
# Display Results
# -----
```

```
cv2.imshow(
    "Original Image",
    gray
)
```

```
cv2.imshow(
    "Original Harris Corners",
    original_result
)
```

```
cv2.imshow(
    "Noisy Image",
    noisy
)
```

```
cv2.imshow(
    "Harris on Noisy Image",
    noisy_result
)
```



```

cv2.imshow(
    "After Gaussian Filtering",
    filtered_result
)

# -----
# Print Results
# -----

print("\n----- RESULTS -----")
print("Original corners:", original_count)
print("Noisy image corners:", noisy_count)
print("After filtering:", filtered_count)

cv2.waitKey(0)
cv2.destroyAllWindows()

```



Result

The experiment shows that noise introduces unwanted intensity variations, which can produce **false corner points**. Gaussian filtering reduces the noise and improves the reliability of Harris corner detection.

3. Hough Transform for Shape Detection

Question

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

Python Code – Hough Line Transform

```
import cv2

import numpy as np

import tkinter as tk

from tkinter import filedialog

# -----

# Select Image

# -----

root = tk.Tk()

root.withdraw()

image_path = filedialog.askopenfilename(

    title="Select Image for Hough Transform",

    filetypes=[

        ("Image Files", "*.jpg *.jpeg *.png *.bmp")

    ]

)

if image_path == "":

    print("No image selected.")

    exit()
```

```
print("Selected image:", image_path)

# -----
# Read Image
# -----

img = cv2.imread(image_path)

if img is None:
    print("ERROR: Image could not be loaded.")
    exit()

print("Image loaded successfully!")

# -----
# Convert to Grayscale
# -----

gray = cv2.cvtColor(
    img,
    cv2.COLOR_BGR2GRAY
)

# -----
# Canny Edge Detection
# -----

edges = cv2.Canny(
    gray,
    50,
    150
)
```

```
# -----  
# Hough Line Transform  
# -----  
lines = cv2.HoughLinesP(  
    edges,  
    rho=1,  
    theta=np.pi / 180,  
    threshold=50,  
    minLineLength=50,  
    maxLineGap=10  
)  
  
# -----  
# Copy Image  
# -----  
result = img.copy()  
  
# -----  
# Draw Detected Lines  
# -----  
if lines is not None:  
  
    print(  
        "Number of lines detected:",  
        len(lines)  
    )  
  
    for line in lines:
```

```

# Works with different OpenCV output shapes
line = np.array(line).flatten()

x1, y1, x2, y2 = line

cv2.line(
    result,
    (int(x1), int(y1)),
    (int(x2), int(y2)),
    (0, 0, 255),
    2
)

else:
    print("No lines detected.")

# -----
# Display Results
# -----

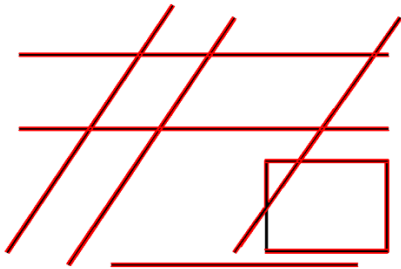
cv2.imshow(
    "Original Image",
    img
)

cv2.imshow(
    "Canny Edge Detection",
    edges
)

```

```
cv2.imshow(  
    "Hough Transform - Detected Lines",  
    result  
)
```

```
cv2.waitKey(0)  
cv2.destroyAllWindows()
```



Result

The program first detects edges using **Canny Edge Detection** and then applies the **Hough Line Transform**. Detected lines are displayed in red.